Data-Centric Model for Architecture's Vulnerabilities Analysis

Michel Bourdellès
Université Bretagne Sud
F-56000 Vannes, France
michel.bourdelles@univ-ubs.fr

Jamal El Hachem
IRISA – UMR 6074 , Univ. Bretagne Sud
F-56000 Vannes, France
jamal.el-hachem@irisa.fr

Salah Sadou
IRISA – UMR 6074 , Univ. Bretagne Sud
F-56000 Vannes, France
salah.sadou@univ-ubs.fr

2025 IEEE 22nd International Conference on Software Architecture (ICSA) | 979-8-3315-2090-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSA65012.2025.00023

***Abstract*—Sensitive data confidentiality and integrity has critical impact on system's security. To handle the inherent complexity of securing sensitive data, protection measures are usually implemented after identifying potential system's vulnerabilities leading to security breaches. At early stages of the system development life cycle, this requires analyzing the various confidentiality levels within the system design, based on data flow representations, to assess and validate compliance with the system's specifications and security requirements. Thus, in this paper we propose a novel security-enhanced data-centric model to analyze vulnerabilities and reinforce sensitive date security at the system architecture level. Generally, it formalizes data flows and enrich them with essential security information knowledge. Specifically, to provide automation and preciseness, we propose a model and tool with two key improvements: (1) ability to confront the data security model with the system architecture to detect potential unsecure flaws within the latter, and (2) capability to automatically complement the code with signatures and ciphering security codes. The evaluation shows that the proposed approach outperforms state-of-the-art approaches, responding to the need of sensitive data protection required in CWE-311, CWE-312, CWE-319 and CWE-327, by offering automated and systematic integration of security controls to the system's design, at the requirement elicitation and architecture design stages.**

***Index Terms*—Architecture practices for secure, explainable and trustworthy software, Architecture design and analysis**

I. Introduction

Securing a hardware and software infrastructure of a set of communicating equipment and, in general, of any system manipulating data to be protected is a particularly complex problem. Solving such problem involves a significant number of means to avoid malicious attacks which take advantage of data vulnerabilities in terms of confidentiality, integrity and availability. To design the system global security, it is advantageous to introduce security as early as possible, at the system specification and design phases, to define homogeneous protections/controls to be applied through the whole system engineering process. Moreover, to cover the wide range of possible attacks on a system the specification of any system cannot be done without a prior risk analysis based on standards as ISO 27005 [20], NIST SP 800-30 [19] or EBIOS-RM [4]. This analysis makes it possible to identify the assets to be secured, the attack likelihood, and determine the appropriate security measures to protect the system. Proposing design solutions to mitigate these identified potential attacks should be covered by the system engineering activities.

The process described in this paper is intended for security engineers in charge of integrating security requirements into the system design, in order to allow automatic analysis of design choices and potential corrective actions for their compliance, as well as their validation at the end of the product development process through automatic security code production. The latter analysis and code ensure the full use of the encryption protection capabilities offered by a deployment platform, independently and in complete confidentiality with respect to the operational implementation of the system. To the best of our knowledge, there is no research or industrial tools enabling these automatic productions.

Recent approaches [34] [26] [25] have been proposed to capture security risk information within a system design model. Our work is aligned with business risk analysis standards and go beyond existing work to formalize risk analysis results, reporting and remediation. It differs from previous work by focusing on data (data-centric security) to reach security conformity in terms of confidentiality and integrity.

To handle the inherent complexity of ensuring security at early design stages, the proposed data-centric process is intended to be exhaustive towards the implementation of security coutermeasures, by exploiting the information captured in system engineering design models such as sensitive data design flows, named Securized Functional Chains (SFC), including labels coming out of the deployed platform security capabilities. Unlike other work [2] [33] based on system analysis data flow representations, our approach use system design information to generate security directives and security code allowing the validation of the system assets security requirements as defined in the specifications. The proposed process adapts to any system modeling (SysML, Cameo Systems Modeller, Arcadia, etc.) exploiting the physical view used in industrial projects, with annotation of information from risk analysis. A model transformation, as proposed by [29] and [22] for security analysis, makes it possible to produce its representation in the form of a list of SFCs. In addition, the security code analysis and the produced software bricks could be easily integrated into an existing system operational development process.

We have implemented in a prototype tool the analysis of systems described as SFCs allowing to provide two types of security analysis, first the compliancy of the design with respect to security requirements, and then the automatic generation

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

of security code for a validation of security requirements, in addition to the validation of operational requirements.

Our contributions are summarized as follows:

- We proposed an extended data flow analysis with information generated from risk analysis to detect security vulnerabilities in the system design.
- We implemented the analysis in a tool prototype.
- We applied the analysis on a dataset of stand-alone SFCs and SFCs generated from Capella industrial tool designs.

To facilitate future research, we have released our studied dataset and prototype [8].

The structure of the paper is defined as follows. In section II, we illustrate the problem statement with an example. In section III, we present our solution, introduced by the description of the general approach, then the formal model allowing the analysis in the form of a list of secure functional chains, and finally the different steps allowing the production of security directives and codes from this description. In section III we also show how our solution is applicable to the composition of subsystems with hidden internal behaviors. We present in section IV the solution application results on examples. Research perspectives are presented in section V. We compare our work with similar works in section VI and conclude in section VII.

II. Problem Statement

This paper tackles the challenge of security requirements elicitation, their exploitation and validation in the system product design process, and the reinforcement of their compliance at the architecture design phase. We illustrate the security challenge in terms of security analysis within a communication system, covering a data-centric need regarding the exchange of information to be secured between two radio devices. This illustrative example will also be used in section IV to illustrate the application of our tooled process and evaluate its effectiveness. This system is described as processing sequences to be deployed on a physical architecture.

As shown in Figure 1, we consider that the data exchanged during conversation has a certain level of sensitivity which requires the implementation of adapted security procedures. During the validation phase, we expect a systematic evaluation, ensuring that this data is accurately protected during its processing. Figure 1 provides examples of potential security related vulnerabilities.

Authors of [9] solved the problem of confidentiality management in a multi-user modeling design sharing elements of different sensitivity levels, with protections guaranteeing the access to the model information even to people who do not have the need to know it throughout the whole product development cycle. Consequently, it is imaginable, in the design of critical systems for which the non-disclosure of sensitive data is a primary requirement, to offer a system design process based on model-based system design.

As far as we know, finely identifying compliance with security constraints is poorly tooled, and requires a tedious analysis effort without assurance of covering all potential

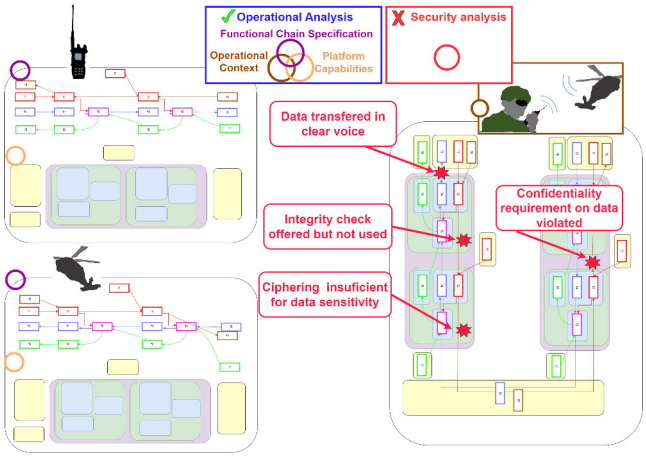


Fig. 1. System design with security flaws.

security flaws. In this paper, we study the ability to enrich system design models with security information and exploit such information to perform security analysis. We therefore propose to supplement the models with information resulting from a security risk analysis. To solve the issue presented in Figure 1, we formalized a process with a corresponding tool allowing to automatically analyze the compliance of system design choices to the stated security requirements, and to automatically produce corresponding corrective security code generated from system modelling design.

III. Proposed Data-Centric Model for Security Analysis

A. General Approach

To gain insights to system architecture vulnerability analysis, we supplement the design flow with security information to assess compliance with security requirements and their validation. The additional information are derived from security risk analysis [25], namely the data assets to be secured and their level of sensitivity. We first supplemented risk analysis results with the processing type: $creation$ of the data flow with sensitivity for the assigned data, $transit$ of encryptable data, $reading$ of data which must not be modified, unlike $write$, and finally $storage$, which can store the data after processing. Moreover, we added the level of protection and encryption services offered by the platform on which the system is deployed. Furthermore, we annotated the sensitivity level assignment policy, namely $maximum$ for the assignment of the data sensitivity level passing through the component or $fixed$ for a given sensitivity level assigned to all data leaving the component.

In what follows, we present in Figure 2 our proposed process steps ② to ⑥ allowing the use of the enriched system design to carry out a systematic analysis, guaranteeing compliance with security requirements during the validation phase. In the next section, we illustrate the results of the analysis on the example from section II, deduced by application of the

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

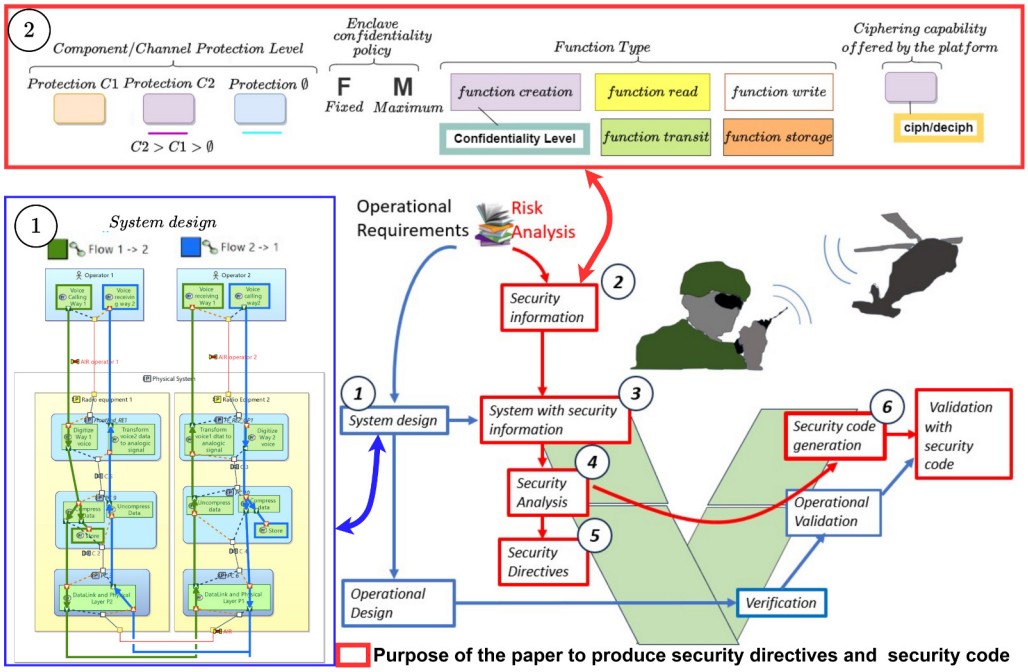


Fig. 2. Integration in an enriched product design flow. Taken up in Figure 8.

processes described in subsection III.C, on $SFCs$ as defined in subsection III.B.

B. Securized Functional Chain (SFC) definition

$SFCs$ are extracted from the Figure 2 step ③ to process security analysis.

We define the set C of confidentiality levels as a set $C : \{C_1, .., C_n, \emptyset\}$. The confidentiality level elements of the set C constitute a lattice with partial order with the relationship $>$ between these elements. In particular $\forall C_i \neq \emptyset, C_i > \emptyset$.

We define the set of Enclave Protection Types as $EPT : \{fixed, maximum\}$. $fixed$ means that any data leaving the enclave with a level of confidentiality protection is assigned this level of confidentiality. $maximal$ means that data leaving the enclave is assigned the maximum level of information confidentiality in this enclave which could affect this data.

We define the set E_c as a list of 3-tuples (e_i, c_j, ept_k) elements of $\{e_1, .., e_m\} \times C \times EPT$, $\{e_1, .., e_m\}$ being enclave identifiers. An e_i enclave offers signature calculation capability if $Sign(e_i) = True$.

An enclave e_i provides encryption/decryption capacity for confidentiality level c_j if $Ciph(e_i, c_j) = True$. For an enclave e, its level of confidentiality protection is denoted $\underline{e}$. We subsequently consider that $Ciph(e, \underline{e}) = True$ for the level of confidentiality protection of the enclaves for which $EPT = fixed$. We also consider that $Ciph(e, c) = true$ with $c \neq \emptyset$ implies $Sign(e) = true$.

We now define the set of function types F_{type} as F_{type} : $\{\textbf{Creation} \times C, \textbf{in read}, \textbf{in write}, \textbf{in storage}, \textbf{in transit}\}$.

A function f is defined by its function type, by the enclave in which it is processed and by an identifier. The set of functions F is a set of 3-tuples $(f_{id}, E_{cid}, f_{type})$. $F : \{fid_1, ..., fid_p\} \times E_c \times F_{type}$.

SFC definition : We finally define the functional chain annotated with security information sfc as a directed graph for which the nodes are functions of F. This graph has a single node of type $creation$ from which it is possible to access the other nodes of the graph by traversing its paths.

System definition : A system to be secured S consists of a set of functional chains, $S : \{sfc_1, ..., sfc_n\}$. The system securing process applies to a system described as a system to be secured S.

Figure 3 presents a set of $SFCs$ system example.

C. Four steps process for security analysis

We built on top of Figure 2 step ④, the following steps to generate security directives ⑤ and code ⑥. A system described as a set of $SFCs$ is presented in Figure 3-A.

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

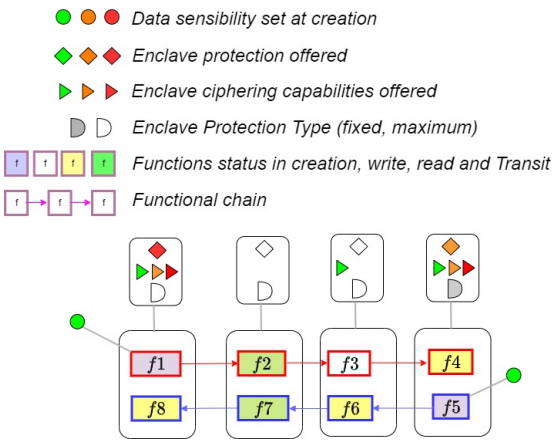


Fig. 3. System as a set of $SFCs$

1) Step 1 - Calculation of integrity check guidelines:
This calculation consists of identifying the path portions of the sfc of the system, which, for the first and last nodes of the path portion, have enclaves which allow the calculation of data signatures, with all the intermediate nodes being of type $in\,read$ or $in\,transit$.
We associate with each node of the sfc, two Boolean variables ic and aic initialized to $false$. $ic = true$ indicates that there is a path for which integrity checking is possible. It does not indicate that all sfc paths passing through this node could have previously been signed. It is the variable aic which will carry this information. We define the calculation of this Boolean variable as follows:
We initialize $aic = true$ for the nodes for which $ic = true$. For each node for which $ic = true$, we traverse the steps of the sfc in the direction of the directed graph and backwards in the direction of the directed graph to the node for which $ic = false$. If in one of these paths none of the nodes f have sign(f)=true in the backward and forward directions, then $aic = false$. Figure 4 describes one example of results of calculating the ic and aic values on a system.

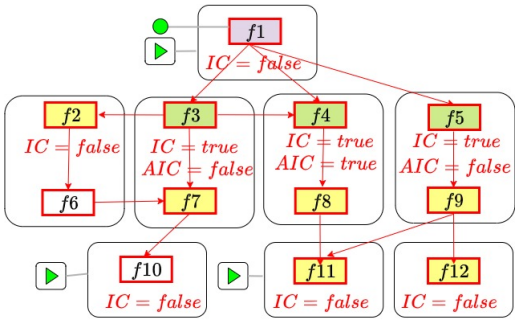


Fig. 4. ic and aic computation examples

2) Step 2 - Calculation of the confidentiality enclave levels crossed by the sfc of the system:

This calculation consists of identifying the confidentiality contamination impact of sfc between them when they share the same processing units, namely the enclaves. It is necessary to provide for the possibility of disclosing data information from one sfc accessible in an enclave by data from another sfc and resulting from write processing by a function in this enclave.

On the other hand, it is also necessary to verify that the level of protection offered by an enclave is greater than the need for protection relating to the level of confidentiality of the data of the sfc carrying out processing on these enclaves.

We describe the calculations to be carried out in this process as inference rules to be applied to the sfc nodes of the system in order to calculate, on the one hand the level of effective protection need $\underline{e_{ef}}$ for a given enclave e, with regard to the processing of functions of sfc on data on the enclave on the other hand the level of confidentiality data from sfc at the output of functions.

The inference rules are expressed as follows:

$$\frac{Axiom\ on\ E\ and\ E_{type} \vdash sfc\ node\ information\ receipt}{Impact\ on\ \underline{E_{ef}} \vdash Impact\ on\ \underline{f}}$$

- $Axiom\ on\ E\ and\ E_{type}$ present the characteristics attached to an enclave and are invariant.
- $Information\ receiving\ from\ node\ of\ sfc$ presents the function to be processed upon reception of the data received as input.
- $Impact\ on\ \underline{E_{ef}}$ calculates the effective protection level of the enclave to be respected to ensure the protection of data passing through this enclave. The value of $\underline{f}$ here is the value of the input confidentiality level of the data to be processed.
- $Impact\ on\ \underline{f}$ calculates the confidentiality level of the data after processing by f in the enclave. This will be the level of confidentiality at the input of the data for the next node of the sfc. $true$ indicates data transmission with a level of confidentiality identical to that of the data received. When it comes to functions receiving and transmitting a stream, $true$ is equivalent to $\underline{f} : \underline{f}$.

We initialize the sfc nodes of the system with the following rules:

- **Initialization rule**: Initialization on enclaves. this rule is independent of sfc and initializes the enclaves.
- **Rule 1:** Rule applied to all steps of functional chains in the case where $E_{type} = fixed$. The $\oplus$ operator applied to the confidentiality levels has the meaning given in [13]. **AnyType** means that the rule applies regardless of the function type.
- **Rule 2:** Rule applied to any step of sfc for which $aic = false$, i.e. at least one path for which it is not possible to carry out a check of integrity of incoming data.
- **Rule 3:** Rule for which whatever the execution steps since the creation of the data relating to the sfc, the data received as input to the node f is always signed and

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

without modification since the signature on the one hand, and with this unmodified data whose integrity check will be carried out in the processing suite on the sfc.

Modifications relating to transit functions will be made in the following section, depending on whether the data in transit can be encrypted or not. For the moment rule 4 is applied there.

- **Rule 4:** Rule on processing for functions in transit, before calculating encryption capabilities.

$$\frac{E \vdash true}{\underline{E_{ef}} : \emptyset \vdash true}\ (R\ init) \qquad \frac{E,\ E_{type} : fixed \vdash f\ AnyType}{\underline{E_{ef}} : \underline{E_{ef}} \oplus \underline{f} \vdash \underline{f} : \underline{E}}\ (R1)$$

$$\frac{E,\ E_{type} : max \vdash f\ AnyType,\ aic = false}{\underline{E_{ef}} : \underline{E_{ef}} \oplus \underline{f} \vdash \underline{f} : \underline{E_{ef}}}\ (R2)$$

$$\frac{E,\ E_{type} : max, \vdash f\ Read,\ aic = true}{\underline{E_{ef}} : \underline{E_{ef}} \oplus \underline{f} \vdash \underline{f} : true}\ (R3)$$

$$\frac{E,\ E_{type} : max, \vdash f\ transit,\ aic = true}{\vdash true}\ (R4)$$

3) Step 3 - Calculation of encryption and decryption directives:

We present here the description elements serving in the proposed process to annotate the encryption and decryption capabilities for a given enclave. To do so, we exploit the information from the sfc annotated with integrity check information in the form of a Kripke structure in order to express the encryption capabilities as a satisfaction result on a state of this structure expressed by a temporal logic property described below.

Firstly, we complete the calculation of the information attached to the sfc nodes by calculating, for each confidentiality level c, $cmc(c)$ for encryption storage capacity. This calculation is carried out by a backward traversal of the directed graph from the states tagged $aic = true$ to the states for which $aic = false$. All these states are initialized $\forall c \in C, cmc(c) = true$. If in none of these paths no node has $ciph(c) = true$, then $cmc(c) = false$. This computation is illustrated in Figure 5-A.

Secondly, we group together all the states attached to a sfc enclave, then divide them into as many states of the kripke structure as distinct tree parts. If there are several functions for a state of the kripke structure, we retain as attached type that of $create$ if there is a state of this type, then $write$ if there is a $read$ function, then $read$ if there is a $read$ function, then $transit$ if there are only transit functions. This type is memorized for a state of the Kripke structure by the variable $type$. the level of confidentiality output from the state of the Kripke structure is memorized by the variable $conf$. A state of the Kripke structure also memorizes the ic information of the initial enclave of the sfc from which it is deduced. Figure 5-B illustrates these transformation from set of $SFCs$ to the Kripke structure representation.

Encryption is only possible on this structure if states satisfy the Computation Tree Logic (CTL) [11] property $CiphCapProp(c)$. To determine the need for encryption coding, it is necessary to annotate the states of the subtree satisfying a particular subassertion. Information on the values storage $ciph$, $both$, $asReceived$, $ciphered$, $deciph$ and $none$ (by default) of the variable $ActionCiph(c)$ are given in Figure 5-C.

We now complete the inference rules for states in transit:

- **Rule 5:** Node in transit for received data always encrypted for the confidentiality level of f.

The information $(ciph(f, c)) \implies \underline{f} : \emptyset$ indicates that the data was received encrypted at the enclave entry and that this data was encrypted with a level of confidentiality protection of c gives this encrypted data a $\emptyset$ confidentiality level.

$$\frac{E \vdash f\ Transit,\ ActionCiph(c) = ciphered, c >= \underline{f}}{\vdash (ciph(f, c)) \implies \underline{f}}(R5)$$

- **Rule 6:** Node in transit for data received always encrypted but for a confidentiality level c lower than f.

In Rule 6, the data is encrypted but not sufficiently to consider that the level of protection of the data to be encrypted can be $\emptyset$. The confidentiality level of f is that of the data before encryption.

- **Rule 7:** Node in transit for received data that cannot be ciphered.

$$\frac{E \vdash f\ Transit,\ ActionCiph(c) = ciphered, \underline{f} > c}{\underline{E_{ef}} : \underline{E_{ef}} \oplus \underline{f} \vdash true}\ (R6)$$

$$\frac{E,\ E_{type} : max \vdash f\ Transit,\ ActionCiph(c) \neq ciphered}{\underline{E_{ef}} : \underline{E_{ef}} \oplus \underline{f} \vdash true}\ (R7)$$

4) Step 4 - Production of system security verification directives and code:

We deduce, in Figure 6, the security and code generation instructions from one Kripke state to another in relation to the labeling of these states, for one level of platform ciphering capability. Red labelling is added to be used as metric coverage in Figure 10.b. The proof of correction these directives and codes are based on is the analysis of the graph paths and the subgraph of the states validating the CTL properties. The results of application on the Figure 3 example are represented in Figure 7.

D. Systems composition security analysis

We have shown that it is possible, when designing a system, by enriching the design model with information from a risk analysis, to: (1) analyze the compliance of security requirements on the design choices to be made, (2) to transmit

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

security directives, and (3) to produce information allowing the generation of necessary security code.

We apply the solution to securing complex critical systems which leads to performing it in the composition of systems respecting the principle of need to know to third party systems to be integrated.

1) Systems composition:

The engineering work of developing software products requires in most cases to integrate part of the software system to be designed with other systems. The functional chains of sensitive data to be secured are rarely specific to the system to be created but pass through several systems. Simply ensuring security on each of the subsystems is not enough to secure inter-system data flows. It is therefore important to be able to carry out the security analysis on all these systems rather then on each individual system. Engineering complex systems incorporate dozens of teams, each working on a particular subsystem. Even though each team can carry out security analysis on its own subsystem, it is crucial to repeat the security analysis on the overall system.

However, the description model in secure functional chains, for a set of interconnected systems, is naturally deduced by interconnecting secure subsystems functional chains. The next challenge is therefore to identify the data flows to be secured on the resulting model composed of the sub-models, and carry out the security analysis on this global model, without the need for additional design analysis at each sub-system level.

We propose in Figure 8.a a composition of systems SFCs obtained by interconnecting their interfaces. The security analysis can then be carried out on the SFC resulting from this composition.

2) Analysis with systems internal hidden design:

The analysis of the previously presented overall system assumes that each of the subsystems is provided with a detailed description of its design and annotated with safety information. Yet, knowing that in real life these subsystems are provided by different industrial partners wanting to protect their know-how, as well as components on the shelves for which providing the detailed description of the security design is antithetical to the need to not disclosure. Consequently, to enable disclosure-preserving analysis of internal system design information, it is necessary to determine the strict minimum information needed for security analysis.

To ensure this, our approach allow to only show data flows between the inputs and outputs, with the minimum information of the other flows, in order to respect a relationship $\sim$ between the initial SFC and the simplified SFC guaranteeing equivalence security analysis $\sim_{Analysis\ result}$ of system composition with the initial SFC and the simplified SFC.

Another solution could be, to grant to subsystem designers only the view on the interfaces disclosed for an interconnection with constraints on the initial level of sensitivity authorized at the input, and the level of sensitivity produced in disclosed output interfaces of this component. This information can be provided as a set of output functions against input stream information, indicating whether the combination is valid, and the result as encrypted or unencrypted data with a given sensitivity level. In the case of encryption, it must be specified whether it is an internal encryption of the subcomponent or an encryption of input data. This also assumes that a trusted authority certifies compliance with these constraints, and that the system integrating this component scrupulously respects the flow constraints to be transmitted and the sensitivity levels of these flows. Figure 8.b presents the two solutions to manage the internal hidden behaviors.

IV. Application

We implemented the process described in section III, and validated it on a set of data made available for reproducibility purposes. Implementation elements are presented in what

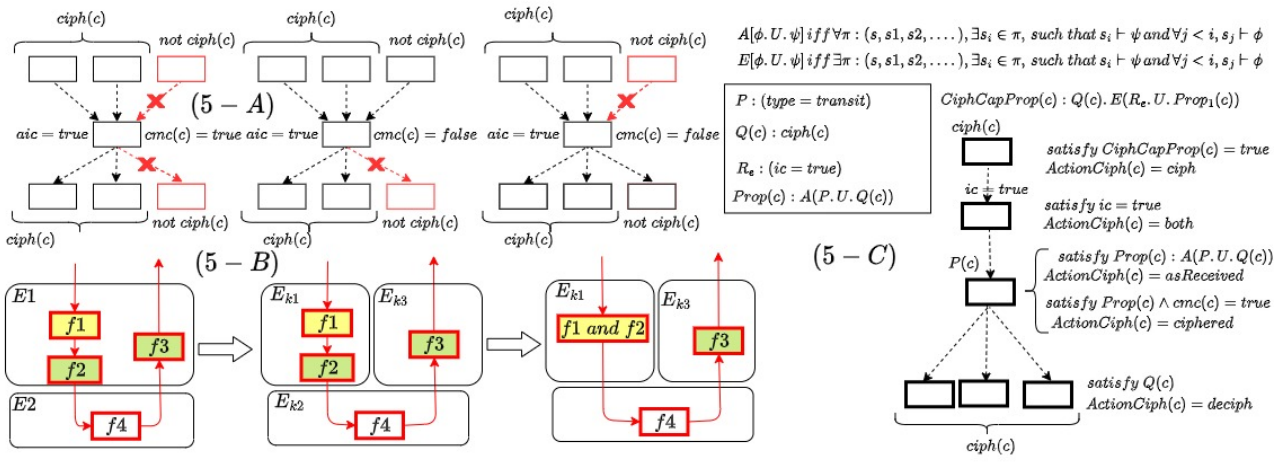


Fig. 5. cmc values computation, transformation to generate the Kripke structure representation and $ActionCiph(c)$ values computation

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

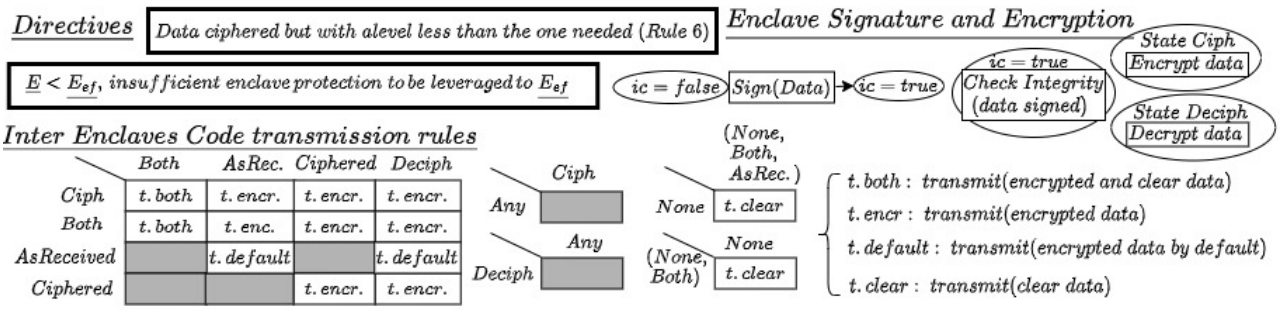


Fig. 6. Security directives and security code generation.

follows, as well as the results from the proposed approach performance on the previously illustrated example.

A. Analysis tool Description

From a system design description annotated with security information, the tool [1] displays the initial information of the parsed SFCs, performs the security analysis as described in section III and displays the result of this analysis as guidance for security requirement elicitation and security code generation.

To facilitate the transfer of our approach to industrial projects, this demonstrator directly uses formats from MBSE tools designed and used by industrials[2] (Capella [1]), with integration into the system design methodology (Arcadia [35]). The SFCs parsed from the input formats are manipulated as instances of the metamodel described in Figure 9.

[1]https://zenodo.org/records/14508184
[2]https://mbse-capella.org

B. Tool Analysis Results

1) Analysis Results on Paper Examples: To illustrate the vulnerability analysis presented in section III and implemented in the tool, we present in Figure 10.a the results of the application on the system presented in Figure 3, with resulting security directives and corrective code (blue flow). In this Figure 10.a, we also present the analysis results on the Capella modelling tool example presented in section II (red flow). We note that model transformations from Capella designs to SFCs are out of the scope of the paper.

2) Analysis Results on a Dataset: As indicated in section III, the proposed approach covers (i) analyzing path graphs and state subgraphs to validate CTL properties on finite state automata, and (ii) theocratically proving the correctness of the algorithms. We note that the performed validation is empirical aiming to identify the limitations in capturing the specification needs of real systems, as detailed in the next section.

All the examples in the dataset are divided between two

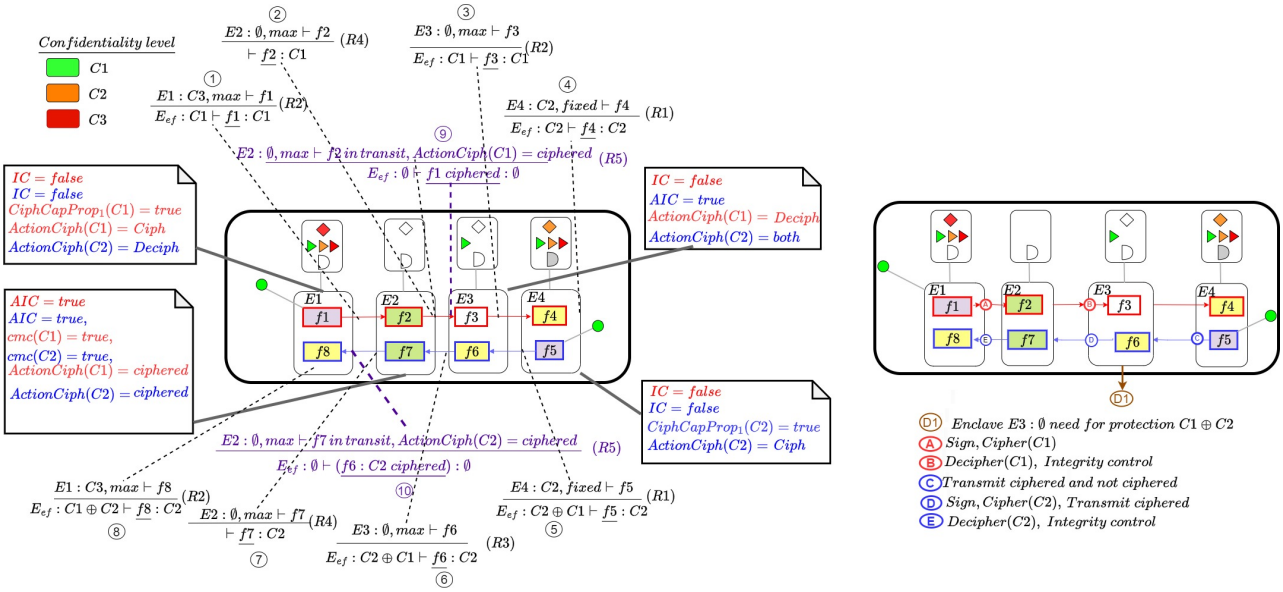


Fig. 7. Security directives and code generation on the Figure 3 system

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

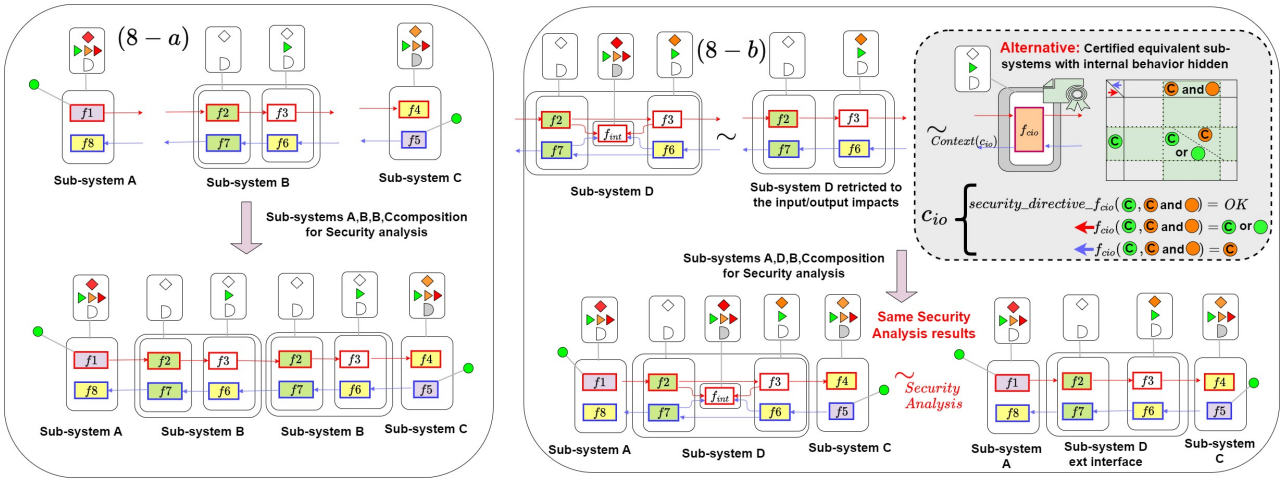


Fig. 8. Security assets confidentiality protection in system design

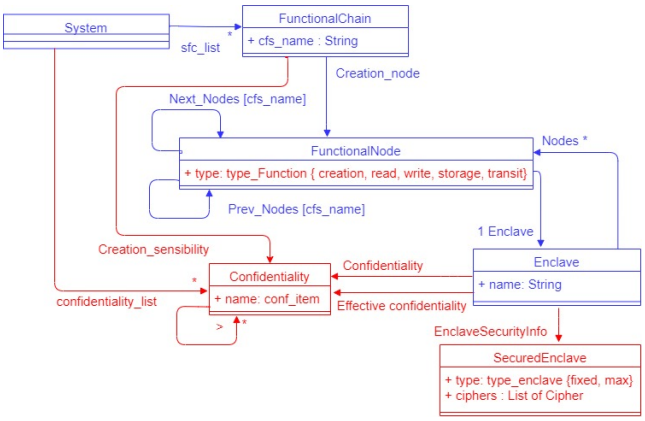


Fig. 9. Metamodel of the SFCs representations

test sets. The first test set is a structural test including the execution of all the formalized rules as the completion metric, as well as all the transitions described in Figure 6, and the second test set is a functional test including the analysis of security protection effectiveness on oracle data [6], presented in Figure 10.b.

For the second tests set, as there is no pre-existing set of tests, we propose the use of known architecture classes (Three-Tier [17], UAV communication network [26]). Results are presented in Figure 11 for separated, composed, and composed with their hidden computed representations.

V. Discussion and Research Perspectives

The proposed approach responds, among others, to the need for protection of MITRE Common Weaknesses Enumerations noted CWE-311 and CWE-319 relating to the absence of encryption protection for sensitive data or insufficient level of encryption protection available CWE-327, as well as CWE-312 relating to the lack of sensitive data storage protection. It offers automatic and systematic integration of security services in systems conceptions from security requirements elicitation, and meets needs as identified for adoption for example of Intent-Based Networking [23] [24] designs.

This work opens new research perspectives that we are investigating. Calls of ciphering functions assume a sharing of keys between pairs of components potentially in different sub-systems. A future ongoing work will focus on the exploitation of platform cryptography devices [12] [30] in the definition of a security platform service for a secure transfer of keys.

The proposed approach is static and assumes the knowledge in the system specification phase of all sensitive data flows. We are also working on its extension to address the implementation of on-the-fly data flows. The combination of real-time analysis and security [16] [10] [15], both working on data flow processes on processing units is also a promising research topic studied.

The presented work can also be extended to automate and refine the messages transmitted to a Security Information Event Management (SIEM) system to identify security requirement violations and the reaction procedures to be applied. This work can also automate the validation tests of security audits [27] and provide evidence for the evaluation of security control applications.

Another work perspective is the upgrade of the demonstrator in terms of Technological Readiness Level (TRL), with an extension to the specialization of data flows, a finer characterization of inter-flow contamination and the use of composite data with different sensitivity and data integrity [7] levels. Moreover, an automated analysis to recommend the more adapted deployment of functional design on different possible physical platforms can also be studied.

VI. Related Work

Our work, as all existing approaches to analyze system confidentiality levels based on data flow representations, benefit from the founding work of [14], [36], [31], here applied

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

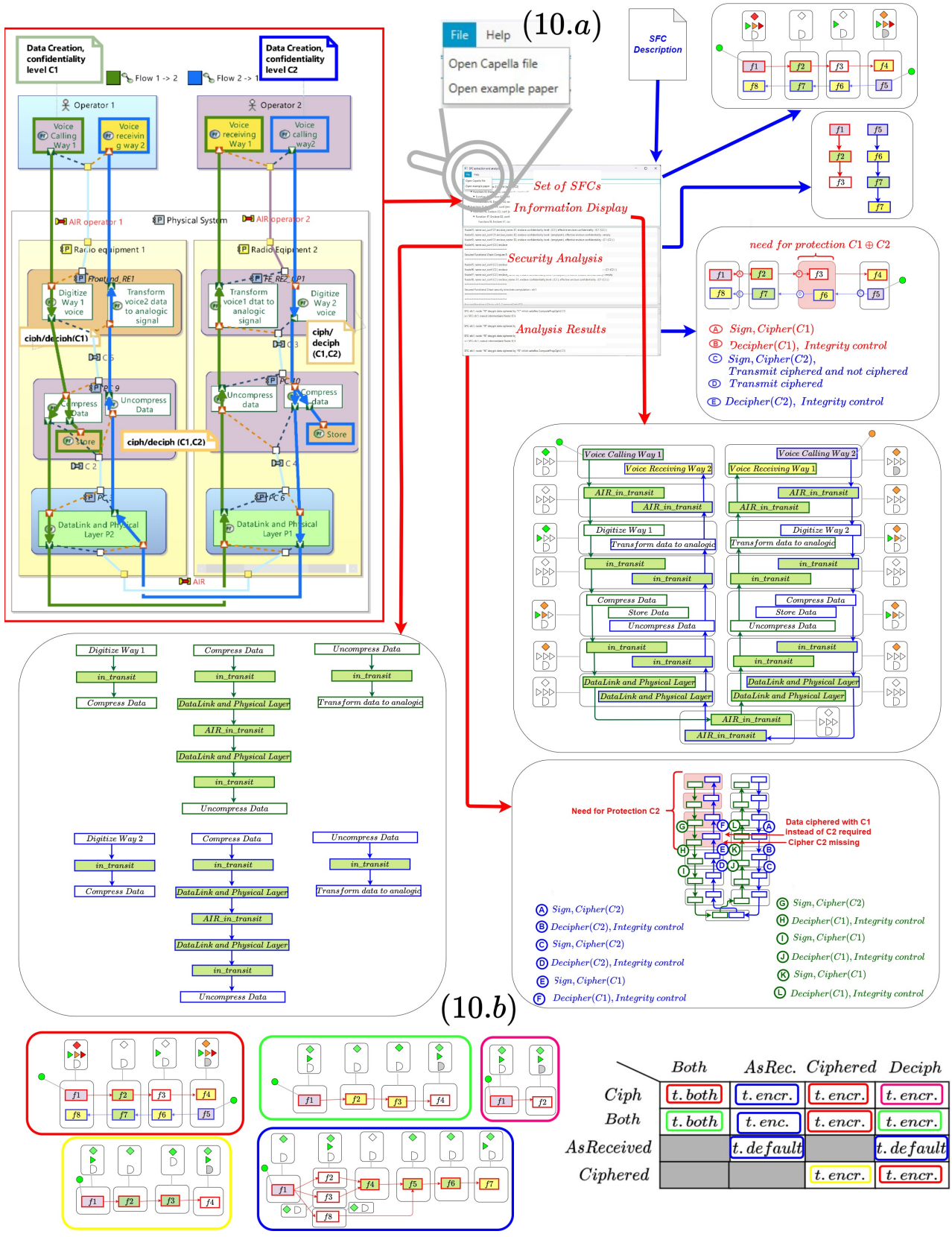


Fig. 10. Application of the approach on the examples presented in Figures 2 and 3 and coverage verification of the transition Table of Figure 6.

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

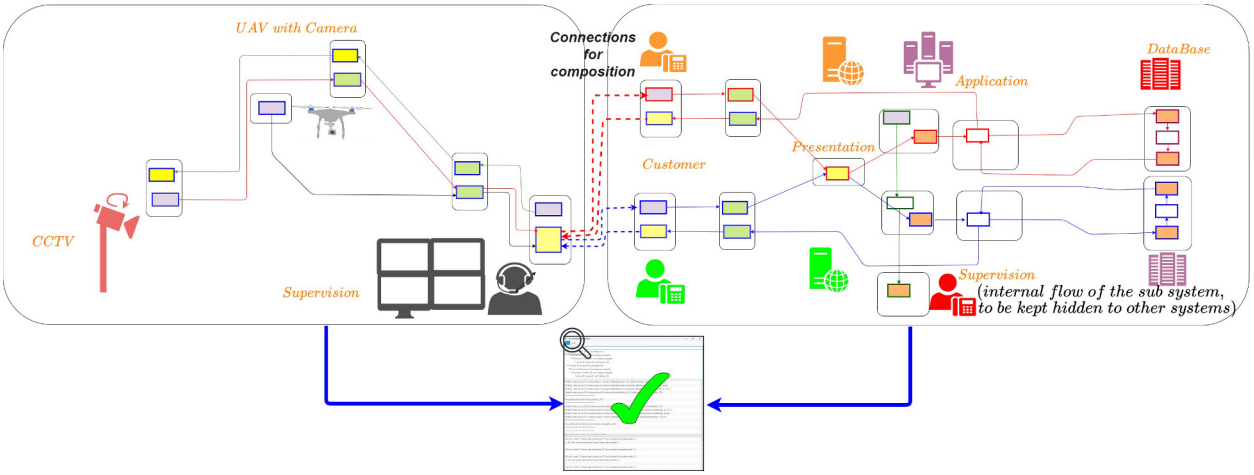


Fig. 11. Three-Tier and UAV network SFCs Security Architecture examples, analysed along with their composition, with internal behaviors hidden.

to systems with enclaves including sensitivity protection and ciphering capabilities.

We may cite [2] which offers a toolbox called CIF (Component Information Flow) in order to implement Information Flow Control [31] in real systems based on a static analysis and code annotation approach, and not based on a system design as obtained with modelling tools such as Capella. The proposed approach combines intra-component and then inter-component processing. Intra-component is based on a static analysis of the code to define confidentiality propagation in the code whereas inter-component processing checks the verification with security policies. Inter-component analysis checks consistency with defined confidentiality and integrity rules defined by user-files, allowing corresponding security code generation. In case of security violation on confidentiality level, a controller can downgrade priviledges, which is a design choice and does not result from a system analysis. Our approach differs in avoiding such priviledge downgrading, and also in analysing the system security as a whole and not at component level whose the composition will satisfy the security of the system. Moreover, in [2] components are software components and not hardware with specific confidentiality policies and sensitivity enclave protection.

Recent works [33], [5], [28], [3] annotate and evaluate Data Flow Diagrams (DFD), to protect the privacy of the software applications modelled. These software application modelling level approaches do not model the security platform capabilities the application is mapped, more easily exploited at system modelling level. This limits to provide the evidence allowing a security assessment satisfying the highest Common Criteria Evaluation Assurance Levels [21]. Moreover, they do not state the separation and confidentiality protection between operational and security codes. The latter, in the design of critical systems, is the responsibility of expert security engineers, integrating for instance security tooled hardware/software platform architectures [18], with security capabilities to be used at most for a secured application mapping.

[29] as well as [22] propose an approach for exploiting annotations of model security information by enriching the SysML and UML metamodels, then verifying them, security requirements refined by transforming models to translate them into input formats of tools such as TTools and SETHEO to perform security analysis. Our work can be exploited as one of these tools that allows performing a security analysis, with capturing information in a SysML-sec or UMLsec model and transformations as a list of sfc, in the same way as with the Capella tool like presented as an example.

[32] proposes a protection analysis of systems described as instances of DDSA (Data-Driven Software Architecture) with an access control protection analysis. This rule-based design in prolog infers compliance with access control constraints. This work lacks of confidentiality level annotations, enclave confidentiality protection, and security code production.

VII. Conclusion

In this work, we designed a security-enhanced data-centric approach which focuses on detecting architecture design vulnerabilities related to confidentiality and integrity. In general, it detects the vulnerabilities based on a data-centric security model and proposes a security control to avoid them and comply to the specified security requirements. Specifically, two techniques are proposed for enhancing both the automation and effectiveness of the proposed approach: a formalized and systematic analysis process and a requirement elicitation and corrective code generation tool.

The evaluation demonstrates that the proposed approach can detect relevant vulnerabilities and propose controls to remediate them, which clearly outperforms the state-of-the-art approaches. As future work, we are working on extending SFCs to the integrity protection requirements resulting from the risk analysis.

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Capella. Available from capella site, (2024), https://mbse-capella.org/

[2] Abdellatif, T., Sfaxi, L., Robbana, R., Lakhnech, Y.: Automating information flow control in component-based distributed systems. In: Proceedings of the 14th International ACM Sigsoft Symposium on Component Based Software Engineering. p. 73–82 (2011). https://doi.org/10.1145/2000229.2000241, https://doi.org/10.1145/2000229.2000241

[3] Alshareef, H., Tuma, K., Stucki, S., Schneider, G., Scandariato, R.: Precise analysis of purpose limitation in data flow diagrams. In: Proceedings of the 17th International Conference on Availability, Reliability and Security. ARES '22 (2022), https://doi.org/10.1145/3538969.3539010

[4] ANSSI: Ebios risk manager – the method. Tech. rep. (2019), https://cyber.gouv.fr accessed 2025/01/22

[5] Antignac, T., Scandariato, R., Schneider, G.: Privacy compliance via model transformations. In: 2018 IEEE European Symposium on Security and Privacy Workshops (EuroSPW). pp. 120–126. https://doi.org/10.1109/EuroSPW.2018.00024

[6] Barr, E.T., Harman, M., McMinn, P., Shahbaz, M., Yoo, S.: The oracle problem in software testing: A survey. IEEE Transactions on Software Engineering **41**(5), 507–525 (2015). https://doi.org/10.1109/TSE.2014.2372785

[7] Biba, K.: Integrity considerations for secure computer systems p. 68 (04 1977)

[8] Bourdelles, M.: Secured functional chain extractor analysis, https://zenodo.org/records/14508184

[9] Bourdellès, M., El Hachem, S., Sadou, S.: Confidentiality management in complex systems design. In: 2024 28th International Conference on Engineering of Complex Computer Systems (ICECCS) (2024)

[10] Chéramy, M., Hladik, P.E., Déplanche, A.M.: Simso: A simulation tool to evaluate real-time multiprocessor scheduling algorithms. In: Proc. of the 5th International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems. WATERS (2014)

[11] Clarke, E.M., Emerson, E.A., Sifakis, J.: Model checking: Algorithmic verification and debugging. Commun. ACM **52**(11), 74–84 (nov 2009), https://doi.org/10.1145/1592761.1592781

[12] Costan, V., Devadas, S.: Intel sgx explained. IACR Cryptol. ePrint Arch. **2016**, 86 (2016)

[13] Denning, D.E., Denning, P.J.: Certification of programs for secure information flow. Commun. ACM **20**(7), 504–513 (jul 1977)

[14] Denning, D.E., Lunt, T.F., Schell, R.R., Heckman, M.R., Shockley, W.R.: A multilevel relational data model. IEEE Symposium on Security and Privacy pp. 220–220 (1987)

[15] Dissaux, P., Singhoff, F., Lemarchand, L., Tran, H.N., Atchadam, I.H.: Combined real-time, safety and security model analysis. In: 9th European Congress ERTSS Embedded Real Time Software and System, Toulouse, France (02 2020)

[16] Feiler, P.H., Gluch, D.P., Hudak, J.J.: The architecture analysis & design language (aadl): An introduction (2006)

[17] GabAllah, N., Farrag, I., Khalil, R., Sharara, H., ElBatt, T.: Iot systems with multi-tier, distributed intelligence: From architecture to prototype. Pervasive and Mobile Computing **93**, 101818 (2023). https://doi.org/https://doi.org/10.1016/j.pmcj.2023.101818

[18] Guo, P., Yan, Y., Zhao, Z., Zhang, L., Zhu, C., Dai, Z.: R/b-secarch: A strong isolated soc architecture based on red/black concept for secure and efficient cryptographic services. Microelectronics Journal **142**, 106024 (2023). https://doi.org/https://doi.org/10.1016/j.mejo.2023.106024

[19] Initiative, J.T.F.T.: Guide for conducting risk assessments. Tech. Rep. rev1, NIST (2012), https://doi.org/10.6028/NIST.SP.800-30r1

[20] ISO: Information security, cybersecurity and privacy protection - information security management systems - requirements. ISO 27005:2022, International Organization for Standardization, Geneva, Switzerland (2023)

[21] ISO15408:2022. Standard, International Organization for Standardization,https://www.commoncriteriaportal.org/cc/index.cfm

[22] Jürjens, J.: Sound methods and effective tools for model-based security engineering with uml. Proceedings of the 27th international conference on Software engineering (2005), https://api.semanticscholar.org/CorpusID:6801778

[23] Kim, J., Okhravi, H., Tian, D.J., Ujcich, B.E.: Security challenges of intent-based networking. Commun. ACM **67**(7), 56–65 (jul 2024). https://doi.org/10.1145/3639702, https://doi.org/10.1145/3639702

[24] Leivadeas, A., Falkner, M.: A survey on intent-based networking. IEEE Communications Surveys Tutorials **25**(1), 625–655 (2023). https://doi.org/10.1109/COMST.2022.3215919

[25] Naouar, D., Hachem, J.E., Voirin, J., Foisil, J., Kermarrec, Y.: Towards the integration of cybersecurity risk assessment into model-based requirements engineering. In: 29th IEEE International Requirements Engineering Conference, RE 2021, Notre Dame, IN, USA, September 20-24, 2021. pp. 334–344. IEEE (2021). https://doi.org/10.1109/RE51729.2021.00037, https://doi.org/10.1109/RE51729.2021.00037

[26] Navas, J., Voirin, J.L., Paul, S., Bonnet, S.: Towards the integration of cybersecurity risk assessment into model-based requirements engineering. In: INCOSE International Symposium, Volume 29, Issue 1, Orlando, USA, September 2019. pp. 850–865. Wiley (2019). https://doi.org/10.1109/RE51729.2021.00037, https://doi.org/10.1002/j.2334-5837.2019.00639.x

[27] NIST800-53a: Assessing security and privacy controls for federal information systems and organizations. Tech. Rep. NIST Special Publication (SP) 800-53a, Rev. 5, Includes updates as of January, 2022, National Institute of Standards and Technology, Gaithersburg, MD (2022). https://doi.org/10.6028/NIST.SP.800-53ar4

[28] Pedroza, G., Muntés-Mulero, V., Martín, Y.S., Mockly, G.: A model-based approach to realize privacy and data protection by design. In: 2021 IEEE European Symposium on Security and Privacy Workshops (EuroSPW). pp. 332–339 (2021). https://doi.org/10.1109/EuroSPW54576.2021.00042

[29] Roudier, Y., Apvrille, L.: Sysml-sec: A model driven approach for designing safe and secure systems. In: 2015 3rd International Conference on Model-Driven Engineering and Software Development (MODELSWARD). pp. 655–664 (2015)

[30] Rushby, J.M.: Separation and integration in mils (the mils constitution) (2008)

[31] Sabelfeld, A., Myers, A.C.: Language-based information-flow security. IEEE J.Sel. A. Commun. **21**(1), 5–19 (sep 2006). https://doi.org/10.1109/JSAC.2002.806121, https://doi.org/10.1109/JSAC.2002.806121

[32] Seifermann, S., Heinrich, R., Reussner, R.: Data-driven software architecture for analyzing confidentiality. In: IEEE International Conference on Software Architecture (ICSA). pp. 1–10 (2019). https://doi.org/10.1109/ICSA.2019.00009

[33] Tuma, K., Scandariato, R., Balliu, M.: Flaws in flows: Unveiling design flaws via information flow analysis. pp. 191–200 (03 2019). https://doi.org/10.1109/ICSA.2019.00028

[34] UAF: Unified Architecture Framework (UAF) Domain Metamodel. OMG (2022), https://www.omg.org/spec/UAF/1.2

[35] Voirin, J.L.: In: Model-Based System and Architecture Engineering with the Arcadia Method. Elsevier (2018). https://doi.org/https://doi.org/10.1016/B978-1-78548-169-7.50027-5

[36] Volpano, D.M., Irvine, C.E., Smith, G.: A sound type system for secure flow analysis. J. Comput. Secur. **4**, 167–188 (1996)

Authorized licensed use limited to: UNIVERSITE DE BRETAGNE SUD. Downloaded on September 21,2026 at 16:19:29 UTC from IEEE Xplore. Restrictions apply.